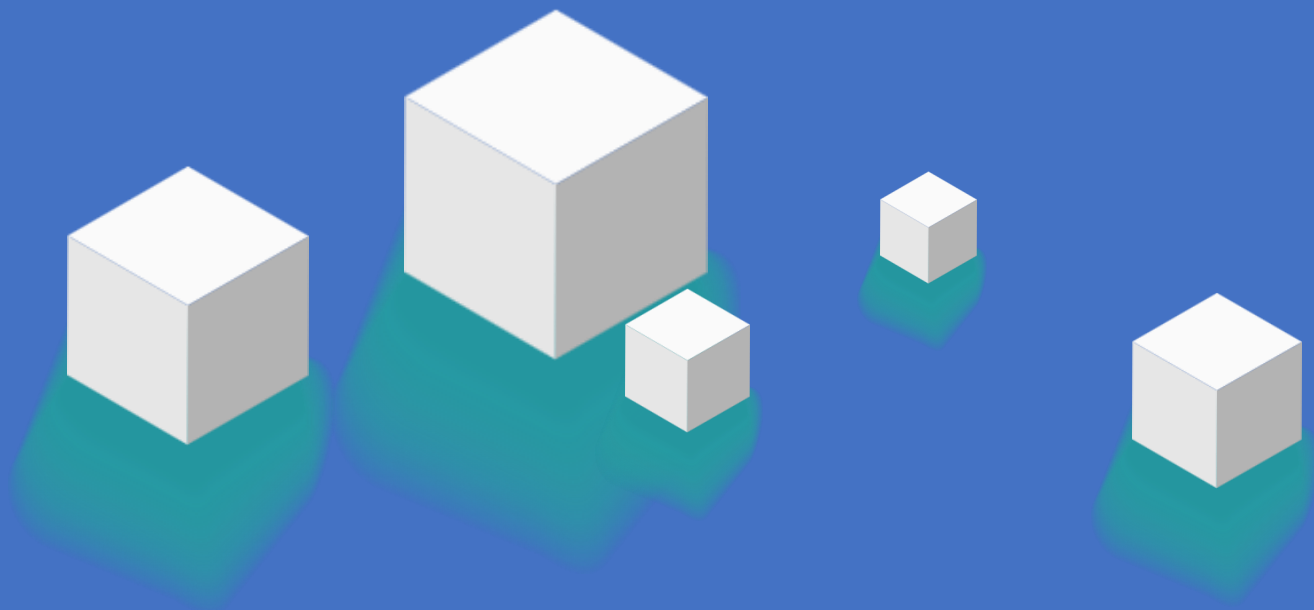


团队架构设计



>> 今天的学习目标

团队架构设计

- Agent VS Workflow
- 团队角色
- 项目结构
- LangGraph 核心五件套
- CASE：交易团队 workflows (LangGraph)
- 场景：风控拦截与重试熔断
- 场景：人在回路真人交互

LangGraph：状态机框架

StateGraph、Node、Edge、Conditional Edge、

Checkpointers



Agent VS Workflow

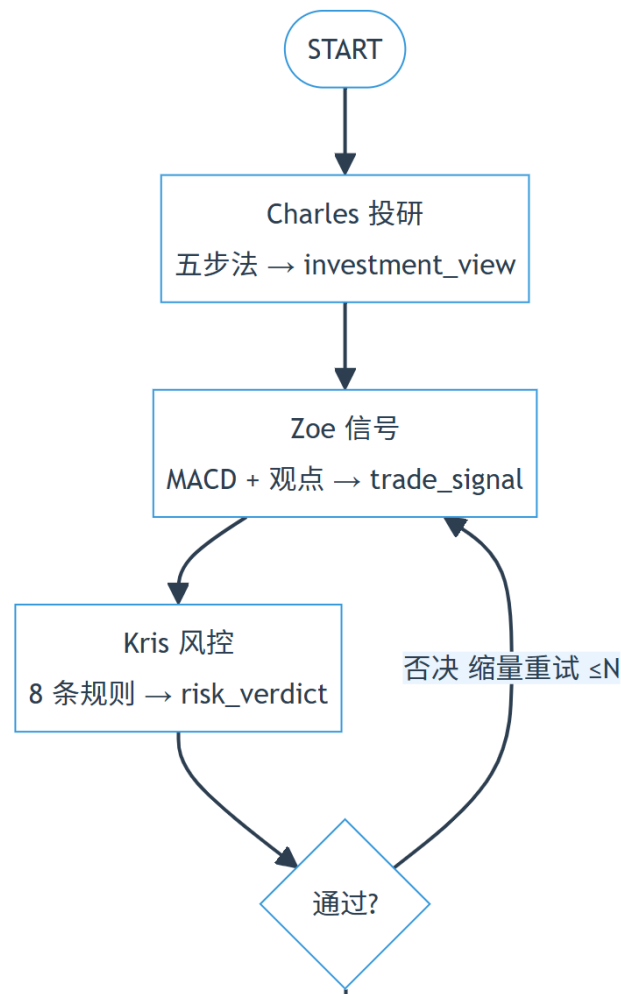
Agent vs Workflow

维度	Agent 模式 (nanobot / deepagents)	Workflow 模式 (LangGraph)
谁决定下一步	LLM 现场决定 (随机性)	代码硬编码 (确定性)
状态共享	靠 prompt 把上下文塞回去	TypedDict State 强约束
条件分支	提示词约束 + LLM 自我判断	Conditional Edge 显式定义
循环 / 重试	基本做不到	Edge 回路 + retry_count
人在回路	提示词约束 (LLM 可能忘)	interrupt + Command(resume)
定时触发	需用户对话驱动	cron / APScheduler 一接就跑
开放式探索	超强	流程一旦定死就僵

- 开放式探索 (写研报 / 自由问答) -> 用 Agent
- 固定流程 (盘前 Pipeline / 风控审批 / 定时巡检) -> 用 Workflow
- 大多数生产场景 -> 用 Hybrid: Workflow 编排 + 节点内是 Agent

团队角色

- Charles 投研情报官: nanobot + 五步法
- Zoe 信号官: MACD, RSI等 + DQN
- Kris 风控官: 8 条核心规则
- Ethan 交易员: 操作MiniQMTTrader



项目结构

CASE-交易团队工作流 (langgraph) /

- └─ main.py # 入口 (单次执行 / 人在回路)
- └─ scheduler.py # APScheduler 定时调度器 (盘前 9:25 自动跑)
- └─ graph.py # StateGraph 编排 + Checkpointer + mermaid 导出
- └─ state.py # TradingState 共享状态契约

└─ nodes/

- | └─ charles_node.py # 投研: 调 deepagents Agent + 输出 HTML 研报
- | └─ zoe_node.py # 信号: subprocess 调 scripts/run_backtest.py
- | └─ kris_node.py # 风控: import lib/risk_engine.py
- | └─ human_node.py # 人在回路: interrupt + Command(resume)
- | └─ trader_node.py # 下单: import lib/miniqmt_trader.py (默认模拟盘)

└─ lib/ # Python 模块

- | └─ risk_engine.py # <- copy from 19-Kris风控体系/1-风控引擎.py
- | └─ miniqmt_trader.py # <- copy from 17-XtQuant实盘交易/4-miniqmt_trader.py

└─ scripts/ # 可执行脚本

- | └─ run_backtest.py # <- copy from 20-AI量化助手(nanobot)/skills/strategy-backtest/
- | └─ get_kline.py # <- copy from 20-AI量化助手(nanobot)/skills/stock-price/

└─ utils/env.py # UTF-8 + Charles 路径注入

└─ outputs/ # 运行产物

└─ workflow.mmd # mermaid 工作流图

└─ reports/ # Charles 生成的 .md / .html 研报附件

└─ runs/ # 每次定时任务的 state JSON

LangGraph 核心五件套

LangGraph: 状态机框架

Thinking: LangGraph的本质是什么?

用一个共享 State 串起来一组函数, 函数之间用边连接, 还能根据 State 走条件分支

与LangChain的区别:

- LangChain: 链式 (Chain) -- 输入 -> 处理 -> 输出, 一条直线, 没有循环
- LangGraph: 图 (Graph) -- 有节点、有边、有循环、有分支, 是个真正的状态机

LangGraph: 状态机框架

Thinking: 什么情况下, 使用 LangGraph?

- 需要多个步骤之间共享状态
- 需要根据某个变量走不同分支
- 需要循环 / 重试 / 回滚
- 需要人在回路
- 需要断点恢复 / 时间旅行

比如人审 interrupt 时进程崩溃, 重启后从同一 checkpoint 续跑, 不必重跑 Charles/Zoe/Kris;

或者审计时从某 thread 的历史快照回到任意一步, 再分叉推演「若当时改判会怎样」。

LangGraph: 核心五件套

Thinking: LangGraph的核心都包括哪些?

- StateGraph: 图本身, 容器
- Node: 一个函数, 读 State 写 State
- Edge: 节点之间的连接
- Conditional Edge: 根据 State 路由
- Checkpointer: State 持久化, 支持 interrupt

State：团队的共享文档

- State 用 TypedDict 定义

State 不是全局变量，它是一份会随节点流动而变化的字典。

在 LangGraph 里推荐用 TypedDict 定义，让字段类型固定：

```
from typing import TypedDict, Optional

class TradingState(TypedDict, total=False):
    stock_code: str
    capital: float
    investment_view: dict    # Charles 写
    trade_signal: dict      # Zoe 写
    risk_verdict: dict      # Kris 写
    approved: Optional[bool] # Human 写
    trade_result: dict      # Trader 写
```

把 State 当成契约：

- 每个字段谁读、谁写，事先约定好，就是节点之间的接口。
- 字段名一旦稳定，节点的具体实现就可以独立替换

比如把 Charles 从 nanobot 换成 qwen agent，下游的 Zoe / Kris 不需要改代码

State: 团队的共享文档

- 节点只返回「这次要改的那一段」，别原地改 state

合并方式和前端 Redux 类似：节点返回一个小字典（只含本次更新的键），由 LangGraph 合并进全局 State；

若在函数里直接改 state 里的键值，可能不会被采纳。

```
def charles_node(state: dict) -> dict:
    view = run_charles(state["stock_code"])
    return {"investment_view": view} # 只返回这次产生的字段

# 错误示范: 不要直接改 state!
def bad_node(state):
    state["investment_view"] = ... # 不可见, 会被 LangGraph 忽略
    return state
```

State：团队的共享文档

- 多节点写同一字段: 用 Annotated reducer

Python 的 Annotated[类型, 归约函数] 在类型旁多挂一个「合并规则」。

LangGraph 根据此进行约定：多个节点都往同一字段写时，用该函数把多份增量合成一份（如 Annotated[list, add] 表示列表按追加合并）。

典型场景是审计日志等多处 append

```
from typing import Annotated
from operator import add

class TradingState(TypedDict, total=False):
    messages: Annotated[list, add] # 多节点 append 自动合并
```

Node：流水线上的工位

在盘前流水线里，投研、信号、风控、人审、下单等每一步就是一个 Node。

代码上它就是一个普通 Python 函数，签名固定为 `state -> 增量`（只返回本次要合并进 State 的那一段，类型上常写作 `partial_state`，即「部分字段」）。它可以：

- 同步函数 / 异步函数都行
- 内部可以调任何东西 -- LLM / SQL / HTTP / shell -- LangGraph 不在意
- 可以是另一个 Agent (这就是 Hybrid 模式)

```
def charles_node(state):  
    agent = create_charles_agent()    # deepagents Agent, 之前写好的  
    result = agent.invoke({  
        "messages": [{"role": "user", "content": state["user_question"]}]  
    })  
    view = parse_view(result)  
    return {"investment_view": view}
```

```
def zoe_node(state):  
    bt = run_macd_backtest(state["stock_code"])  
    direction = decide(state["investment_view"], bt)  
    return {"trade_signal": {"direction": direction, ...}}
```

Charles 节点：内部调 deepagents Agent

Zoe 节点：调 shell 跑回测脚本

Node：流水线上的工位

Thinking: 节点函数为什么这么无聊？

节点函数就是个普通函数

=> 这是 LangGraph 的设计哲学: 框架不绑架你。

节点函数很纯粹, 想做什么完全自由

=> 调 LLM、调本地脚本、调数据库、调 webhook 都行。

Edge: 工位之间的流水线

固定 Edge 很简单:

```
g.add_edge(START, "charles")  
g.add_edge("charles", "zoe")  
g.add_edge("zoe", "kris")  
g.add_edge("trader", END)
```



但实际工作流不会这么直 -- 风控可能否决, 需要回到 Zoe 重生成。

=> 这就要用条件边

Conditional Edge: 条件边

核心: 一个路由函数

路由函数读 State, 返回字符串作为下一节点的名字:

```
def route_after_kris(state) -> str:
    verdict = state.get("risk_verdict", {})
    retry = state.get("retry_count", 0)
    max_retry = state.get("max_retry", 2)

    if verdict["decision"] in ("reject", "halt"):
        if retry >= max_retry:
            return END      # 熔断, 终止
        return "zoe_retry"  # 否则回 Zoe 重发
    return "human"         # 通过则进入人审
```

```
g.add_conditional_edges(
    "kris",
    route_after_kris,
    {"zoe_retry": "zoe_retry", "human": "human", END: END},
)
```

Conditional Edge: 条件边

Thinking: 为什么要单独的 `zoe_retry` 节点?

因为路由函数不能改 `state` -- 它是只读的。

但我们想: 每次重试给 `retry_count` 加 1 => 所以专门造一个微型节点:

```
def zoe_retry_bump(state):  
    return {"retry_count": state.get("retry_count", 0) + 1}  
  
g.add_node("zoe_retry", zoe_retry_bump)  
g.add_edge("zoe_retry", "zoe") # 计数后回到 zoe
```

常见错误: 把 改 `state` 和 路由 混在一个函数里。

LangGraph 强制你分开: 路由函数纯函数 (输入 `state`, 输出 `next_node`); 改 `state` 走真正的 Node。

=> 这种 职责分离 是 LangGraph 编排清晰的秘诀。

Checkpoint + Interrupt: 让人在回路 变得可能

Checkpoint: State 持久化

Checkpoint 是个Key-Value 存储，每次节点执行完，State 都会被存进去。

最简单的是 MemorySaver (内存)，生产可用 SqliteSaver / PostgresSaver。

```
from langgraph.checkpoint.memory import MemorySaver

graph = builder.compile(checkpointer=MemorySaver())

# 用 thread_id 区分不同会话
config = {"configurable": {"thread_id": "trade-001"}}
state = graph.invoke(initial, config=config)
```

Checkpoint + Interrupt: 让人在回路 变得可能

Interrupt: 让节点停下来，等人反馈

关键 API: `interrupt(payload)`。节点内调用它，整个图会暂停，把 `payload` 抛回给上层。

上层收集人类回复后，用 `Command(resume=...)` 继续往下走。

```
from langgraph.types import interrupt, Command

def human_review_node(state):
    payload = {
        "signal": state["trade_signal"],
        "prompt": "是否授权下单? yes/no",
    }
    user_reply = interrupt(payload) # <-- 在这里卡住
    approved = (user_reply.lower() == "yes")
    return {"approved": approved}
```

```
# 调用方
state = graph.invoke(initial, config=config)
# 上面那行执行到 interrupt 时会"返回", 但 state 还停在 human_review

snapshot = graph.get_state(config)
if "human" in snapshot.next:
    user_reply = input("是否授权: ")
    # resume 继续往下走, 直到下一个 interrupt 或 END
    state = graph.invoke(Command(resume=user_reply), config=config)
```

Checkpoint + Interrupt: 让人在回路 变得可能

Thining: 为什么Checkpoint + Interrupt 是 Workflow 模式的杀手特性?

在 Agent 模式 (nanobot/deepagents) 里, 必须人类授权才下单

=> 只能靠 prompt 提醒 LLM (它可能忘)。

在 Workflow 模式里, interrupt 是硬性卡点, 即 没有 Command(resume=...) 它就是不会往下走, state 还会被 Checkpointer 存好, 崩溃重启都能恢复。

=> 机构量化会使用工作流, 因为合规和安全是 硬约束。

20 行代码: 最小可运行 LangGraph

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

class S(TypedDict, total=False):
    n: int
    log: list

def double(s): return {"n": s["n"] * 2, "log": s.get("log", []) + ["x2"]}
def add_one(s): return {"n": s["n"] + 1, "log": s.get("log", []) + ["+1"]}
def route(s): return "add_one" if s["n"] < 100 else END

g = StateGraph(S)
g.add_node("double", double)
g.add_node("add_one", add_one)
g.add_edge(START, "double")
```

```
g.add_conditional_edges("double", route, {"add_one": "add_one", END: END})
g.add_edge("add_one", "double") # 形成循环: 翻倍 -> 加一 -> 翻倍 ...

graph = g.compile(checkpointer=MemorySaver())
result = graph.invoke({"n": 3}, config={"configurable": {"thread_id": "demo"}})
print(result)

{'n': 126, 'log': ['x2', '+1', 'x2', '+1', 'x2', '+1', 'x2', '+1', 'x2']}
```

Summary

麻雀虽小五脏俱全:

- State (TypedDict)
- Node (函数)
- Edge (固定边)
- Conditional Edge (路由)
- Checkpointer (MemorySaver)
- 循环 (add_one -> double 形成 loop)。

CASE：交易团队 workflow (LangGraph)

CASE: 交易团队 workflow

我们之前已经分别造好了量化团队的每个零件:

- Charles (投研情报官)

五步法分析 + 7 个 skills (web-search / read-pdf / financial-analysis ...), 输出结构化研报

- Zoe (信号官)

跑 MACD 回测拿技术面信号, 结合基本面做多因子融合, 算出仓位和价格

- Kris (风控官)

8 条规则 (单笔限额 / 集中度 / ATR 仓位 / 事件关键词 ...), 输出 approve / reject

- Trader (交易执行)

接 miniQMT 下单

CASE: 交易团队 workflows

Thinking: 如何将这4个角色, 串成一条流水线?

- 投研: Charles 分析标的基本面, 给出看多/看空/中性的立场和信心度
- 出单: Zoe 结合 Charles 的观点和 MACD 技术面信号, 决定买/卖/观望, 算出仓位和价格
- 风控: Kris 对 Zoe 的单子跑 8 条风控规则
- 人审: 把三方信息汇总给人类做最终授权
- 执行: Trader 接到授权后下单

这五步有严格的先后依赖 -- Zoe 必须等 Charles 出完研报才能决策, Kris 必须拿到 Zoe 的单子才能审批。
在没有 LangGraph 之前:

- 方案 A: 人工串 -- 手动跑完一个脚本, 复制输出, 粘贴到下一个脚本的输入
- 方案 B: 纯 Agent -- 让LLM 自己决定先调谁、再调谁, 但顺序不可控, 可能跳过风控直接下单

CASE: 交易团队 workflows

TO DO: 以 600519.SH 贵州茅台 为例, 搭建完整的交易团队 workflows

- 资金: 100 万 (足够买茅台一手, $1407 \text{ 元/股} \times 100 \text{ 股} = 14.07 \text{ 万}$)
- 当前市场状态: MACD 技术面持续看跌 (2026-04-21 最新信号), 但 Charles 基本面看多
- 团队预期行为: Charles 看多 -> Zoe 小仓试探出单 -> Kris 风控通过 -> 人审放行 -> Trader提交

LangGraph 的解法: 把五步编排成一条确定性流水线, 每个节点只管自己的事, State 在节点之间自动流转。顺序由图的 Edge 钉死, 不靠 LLM 决定。

CASE: 交易团队 workflow

```
python main.py --stock 600519.SH --capital 1000000 --auto-approve
```

- `--stock 600519.SH` 标的代码 (上交所要带 .SH 后缀)
- `--capital 1000000` 可用资金 100w (足够买茅台一手)
- `--auto-approve` 自动放行人审 (跳过人在回路, 一气跑通)

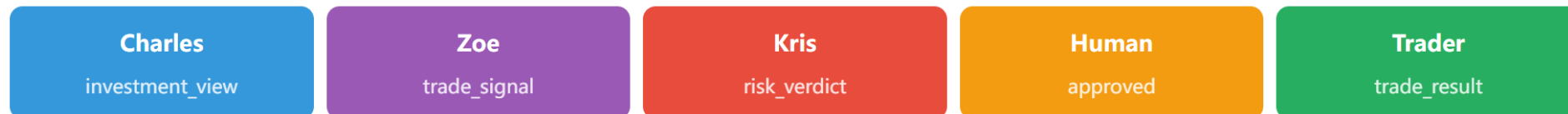
不传 `--auto-approve` 时会卡在人审节点等用户输入 yes/no

LangGraph特性使用

LangGraph 特性	说明
StateGraph	定义包含 5 个节点的有向图
TypedDict State	TradingState 钉死字段所有权
固定 Edge	charles -> zoe -> kris 三段直线
Conditional Edge	kris approve -> human; human approved -> trader (这里都走 happy path)
Hybrid 范式	charles_node 内部是 deepagents Agent, 但调度由 LangGraph 接管

LangGraph特性使用

State 共享流转图



Charles 写: $\rightarrow \text{state["investment_view"]} = \{\text{stance, confidence, summary, catalysts, risks, report_html_path}\}$

Zoe 读: $\leftarrow \text{state["investment_view"]}$ (决策融合: stance + confidence)

Zoe 写: $\rightarrow \text{state["trade_signal"]} = \{\text{direction, quantity, price, reason, ...}\}$

Kris 读: $\leftarrow \text{state["trade_signal"]} + \text{state["investment_view"]}$ (双输入)

Kris 写: $\rightarrow \text{state["risk_verdict"]} = \{\text{decision, is_approved, reason, suggested_max_pct}\}$

Human 读: $\leftarrow \text{state["trade_signal"]} + \text{state["investment_view"]} + \text{state["risk_verdict"]}$ (三方汇总)

Human 写: $\rightarrow \text{state["approved"]} = \text{True/False}$

Trader 读: $\leftarrow \text{state["trade_signal"]} + \text{state["approved"]}$

Trader 写: $\rightarrow \text{state["trade_result"]} = \{\text{dry_run, order_id, note}\}$

- State 在节点之间流动，而不是复制。
- 每个节点只声明它"读什么 / 写什么", LangGraph 自动把更新合并回 State。
- 这种隐式数据流比 Agent模式靠 prompt 传上下文清晰, 也省 token。

代码逻辑：State

1) State 契约定义

```
from typing import TypedDict, Annotated, Optional
from operator import add
```

```
class TradingState(TypedDict, total=False):
```

```
    # === 输入 ===
```

```
    stock_code: str
```

```
    capital: float
```

```
    user_question: str
```

```
    # === 各节点产出 (按"谁写"标注) ===
```

```
    investment_view: InvestmentView # Charles 写
```

```
    trade_signal: TradeSignal      # Zoe 写
```

```
    risk_verdict: RiskVerdict      # Kris 写
```

```
    approved: Optional[bool]       # Human 写
```

```
    trade_result: TradeResult      # Trader 写
```

```
    # === 控制流 ===
```

```
    retry_count: int
```

```
    max_retry: int
```

```
    # === 审计日志 (Annotated reducer 让多节点可 append) ===
```

```
    messages: Annotated[list, add]
```

代码逻辑：Charles节点

2) Charles 节点 (Hybrid: 调 deepagents Agent + 写 HTML 研报)

```
def charles_node(state):
    stock = state["stock_code"]
    question = state["user_question"] + EXTRACT_SUFFIX

    # 复用 create_charles_agent (内部是 deepagents)
    agent = create_charles_agent()
    result = agent.invoke(
        {"messages": [{"role": "user", "content": question}]},
        config={"configurable": {"thread_id": f"trading-team-{stock}"}}
    )
    report_text = _extract_final_text(result)
    view = _extract_view(report_text)    # 解析末尾 ``json`` 摘要
    view["raw_report"] = report_text
```

```
# 把研报落盘成 .md + .html, 路径塞回 state
md_path, html_path = _save_report_attachments(stock, view, report_text)
view["report_md_path"] = md_path
view["report_html_path"] = html_path

return {"investment_view": view, "messages": [...]}
```

Thinking: Charles 节点为什么这么写?

因为之前有相关代码——用 deepagents 包装的强大 Agent, 含五步法 prompt + 7 个 skills (web-search / read-pdf / financial-analysis / ...).

=> 没必要在 LangGraph 里重新实现, 直接 import 用就行。

=> Hybrid 范式: 外层是 Workflow (流程稳定), 节点内是 Agent (能力强大)。

[nodes/charles_node.py](#)

代码逻辑: Zoe 节点

3) Zoe 节点 (调 回测脚本 + 多因子融合)

```
def zoe_node(state):  
    view = state.get("investment_view", {})    # <-- 读 Charles 产出  
    last_verdict = state.get("risk_verdict", {}) # <-- 重试时读 Kris 上一轮  
  
    bt = _run_backtest(state["stock_code"], strategy="macd", count=250)  
    latest_signal = bt["latest_signal"] # golden_cross / bullish / death_cross  
    / bearish (bearish=看跌)  
  
    stance = view.get("stance", "neutral")  
    confidence = float(view.get("confidence", 0.5))  
  
    # 多因子决策矩阵 (基本面 x 技术面)  
    if latest_signal in ("golden_cross", "bullish") and stance == "bullish":  
        direction, pos_pct = "buy", _decide_position_pct(stance, confidence)  
        reason = f"共振: MACD {latest_signal} + Charles bullish"
```

```
        elif stance == "bullish" and confidence >= 0.7 \  
            and latest_signal in ("bearish", "death_cross"):  
            # 基本面强但技术面分歧 -> 半仓试探  
            direction, pos_pct = "buy", _decide_position_pct(stance, confidence) *  
0.5  
            reason = f"分歧: Charles 强势 bullish, MACD {latest_signal} -> 小仓试探"  
        else:  
            direction, pos_pct = "hold", 0.0  
  
    return {"trade_signal": {...}}
```

底层调的回测脚本来自 nanobot 助手项目, 已
copy 到 scripts/ 目录

[nodes/zoe_node.py](#)

代码逻辑：Kris 节点

4) Kris 节点 (import 风控引擎)

```
# 把 lib 加到 sys.path, 直接引入之前的风控引擎
if str(LIB_DIR) not in sys.path:
    sys.path.insert(0, str(LIB_DIR))
from risk_engine import Decision, Order, RiskManager
def kris_node(state):
    signal = state["trade_signal"]
    view = state["investment_view"]
    capital = float(state["capital"])
    atr_value = _fetch_kline_atr(signal["stock_code"])

    kris = RiskManager(config={
        "pre_trade": {"max_order_amount": max(capital * 0.5, 100_000), ...},
        "circuit_breaker": {"max_daily_loss_pct": 0.02},
    })
```

```
kris.start_day(capital)
kris.macro.update_vix(18.5)

portfolio = {"total_asset": capital,
             "prices": {signal["stock_code"]: signal["price"]},
             "atr": {signal["stock_code"]: atr_value}}
order = Order(stock_code=signal["stock_code"],
              direction=signal["direction"],
              amount=signal["price"] * signal["quantity"], price=signal["price"])

# 把 Charles 风险点 + 催化剂拼成"新闻文本", 喂事件关键词检查
news_text = " ".join(view.get("risks", []) + view.get("catalysts", []))
decision = kris.approve(order, portfolio, context={"news_text": news_text})

return {"risk_verdict": {"decision": decision.decision.value, ...}}
```

nodes/kris_node.py

代码逻辑: Trader 节点

5) Trader 节点 (import MiniQMTTrader, 默认 dry-run)

```
def trader_node(state):  
    if not state.get("approved"):  
        return {"trade_result": {"note": "用户未授权"}}  
  
    dry_run = os.environ.get("TRADER_DRY_RUN", "1") == "1"  
    if dry_run:  
        print(f"[DRY-RUN] {direction} {qty} @ {price}")  
        return {"trade_result": {"dry_run": True, ...}}  
  
    # 真实下单: MiniQMTTrader  
    from miniqmt_trader import MiniQMTTrader  
    trader = MiniQMTTrader(qmt_path=os.environ["QMT_PATH"],  
                           account_id=os.environ["ACCOUNT_ID"])
```

```
trader.connect()  
order_id = trader.buy(stock, qty, price=price, strategy_name="team-workflow")  
trader.disconnect()  
return {"trade_result": {"dry_run": False, "order_id": order_id}}
```

代码逻辑：图编排

6) 图编排

```
def build_graph(with_checkpointer=True):  
    g = StateGraph(TradingState)  
    g.add_node("charles", charles_node)  
    g.add_node("zoe", zoe_node)  
    g.add_node("kris", kris_node)  
    g.add_node("human", human_review_node)  
    g.add_node("trader", trader_node)  
  
    g.add_edge(START, "charles")  
    g.add_edge("charles", "zoe")  
    g.add_edge("zoe", "kris")  
    g.add_conditional_edges("kris", route_after_kris, {...})  
    g.add_conditional_edges("human", route_after_human, {...})  
    g.add_edge("trader", END)  
    return g.compile(checkpointer=MemorySaver())
```

CASE：交易团队 workflows

运行结果：

```
# python main.py --stock 600519.SH --capital 1000000 --auto-approve

#####

# 团队 workflows 启动 -- 标的 600519.SH | 资金 1,000,000 | auto-approve=True

#####

=====

[Charles] 投研情报官开始工作 -- 标的: 600519.SH

=====

[Charles] 立场: 看多 | 信心: 0.82

[Charles] 核心: 茅台已从量价驱动转向机制驱动, 直营占比破50%、i茅台成第二主渠道、Q1业绩超预期, 估值应反映机制溢价而非单纯增长性。

[Charles] 催化剂 3 条 | 风险 2 条

[Charles] 研报附件: 600519_SH_20260421_130320.html (HTML),
                    600519_SH_20260421_130320.md (MD)
```

```
=====

[Zoe] 信号官开始工作 -- 标的: 600519.SH

=====

[Zoe] MACD 回测 2025-04-10 ~ 2026-04-21 | 胜率 30.8% | 总收益 -13.91%
      | 最新信号 看跌 @ 1412.69

[Zoe] 已读取 Charles 研报附件:
outputs/reports/600519_SH_20260421_130320.html

[Zoe] Charles 催化剂参考: 2026年4月25日一季报正式发布

[Zoe] Charles 风险提示: 商务宴请复苏不及预期导致批价二次下探

[Zoe] 算出仓位不足 1 手, 按最小试探单 100 股出单 (交给 Kris 审批)

[Zoe] 决策: BUY 100股 @ 1412.69
      | 分歧: Charles 强势看多(0.82), MACD 看跌 -> 小仓试探
```

CASE：交易团队 workflows

=====

[Kris] 风控官开始审批

=====

[Kris] 600519.SH ATR(14) = 22.587

[Kris] [PASS] 综合审批: 所有检查通过

=====

[人在回路] 待用户授权下单:

=====

stock : 600519.SH
direction : buy
quantity : 100
price : 1412.69
amount : 141269.0
charles_stance : bullish
charles_confidence : 0.82

charles_summary : 茅台已从量价驱动转向机制驱动, 直营占比破50%、i茅台成第二主渠道...

zoe_reason : 分歧: Charles 强势 bullish(0.82), MACD bearish -> 小仓试探

zoe_winrate : 30.8

kris_decision : approve

kris_reason : 所有检查通过

[自动模式] 自动回复 yes

=====

[Human] 用户决定: APPROVED

=====

=====

[Trader] 交易执行台

=====

[Trader] [DRY-RUN] BUY 600519.SH 100股 @ 1412.69

CASE：交易团队 workflows

=====

团队 workflows 执行汇总

=====

标的: 600519.SH
可用资金: 1,000,000

[Charles] bullish (信心 0.82) -- 茅台已从量价驱动转向机制驱动, 直营占比破50%...

[Zoe] buy 100 股 @ 1412.69 -- 分歧: Charles 强势 bullish(0.82), MACD bearish -> 小仓试探

[Kris] approve -- 所有检查通过

[Human] approved = True

[Trader] dry-run: buy 100 @ 1412.69

--- 节点对话历史 ---

[13:03:20] charles | bullish (0.82) -- 茅台已从量价驱动转向机制驱动, 直营占比破50%...

[13:03:25] zoe | buy 100股 @ 1412.69 (分歧: Charles 强势 bullish(0.82), MACD bearish -> 小仓试探)

[13:03:30] kris | approve -- 所有检查通过

[13:03:30] human | approved

[13:03:30] trader | dry-run: buy 100 @ 1412.69

=====

Summary

生产的产物:

- 研报 Markdown

outputs/reports/600519_SH_20260421_130320.md => 原稿, 五步法正文

- 研报 HTML

outputs/reports/600519_SH_20260421_130320.html => 带样式渲染, 浏览器直接打开

Summary

节点	耗时	说明
Charles	~168 秒	调 deepagents Agent + 多轮 web_search + 写研报 (本次 LLM 输出 7000+ 字, 五步法分析较深)
Zoe	~5 秒	subprocess 跑回测脚本 (含 xtdata 拉数据)
Kris	~5 秒	subprocess 拉 K 线算 ATR + 8 条规则检查
Human (auto)	~0 秒	auto-approve 直接放行
Trader	~0 秒	dry-run 不连 miniQMT
合计	~178 秒	主要时间花在 Charles 的 LLM 调用 (qwen-plus + 多轮 web_search)

Summary

- State 共享是 Workflow 的硬通货

注意 Zoe 节点的日志: "已读取 Charles 研报附件: ..."

=> 这就是 State 共享。如果用 Agent 模式做同样的事:

方案 A: 把 Charles 整个研报塞进 Zoe 的 prompt -> 几千字, token 暴涨, LLM 还可能忽略

方案 B: 让主 LLM 协调两个 subagent -> 又得多一层 LLM 思考, 又慢又贵

本 CASE: `state["investment_view"]["report_html_path"] = "..."` -> 一个字段读取完事

Summary

- Hybrid 用于生产

Charles 节点内部仍然是 deepagents Agent (200+ 行 prompt + 7 个 skills + 多轮 LLM 调用), 因为"写研报"本身就是开放式任务, Agent 模式是对的。

但 什么时候写、写完给谁、出错怎么办, 不该让 LLM 决定 => 这是 Workflow 的事。

=> 这种"外层 Workflow + 内层 Agent"就是 Hybrid 范式

=> 机构 AI 系统的标准姿势。

打卡：交易团队 workflows



以你的自选股为例，资金 100 万、自动放行人审为例：

把团队按固定顺序从 Charles 跑到 Trader，一气跑通整条流水线：

- Charles 投研：用国泰君安五步法分析 600519，给出立场（看多 / 看空 / 中性）和信心度。
- Zoe 信号：跑 MACD 回测，结合 Charles 的观点，决定买 / 卖 / 观望、出价和数量。
- Kris 风控：对 Zoe 的单子跑 8 条核心规则，输出 approve / warn / reject。
- 人在回路：把 Charles + Zoe + Kris 三方信息汇总，等待人类授权；本场景由 --auto-approve 自动回 yes。
- Trader 执行：拿到授权后下单（默认模拟盘），全流程结束。

场景： 风控拦截与重试熔断

场景：风控拦截与重试熔断

TO DO: 以 600519.SH (贵州茅台)为例进行操作

- 资金: 20 万 (故意不够) -- 茅台一手 14.07 万, Kris 单笔金额上限 = 资金 50% = 10 万
- 当前市场状态: 跟 CASE-A 一样, MACD death_cross + Charles bullish
- 预期行为:

Charles 看多 -> Zoe 出 100 股试探单 (140,724 元)

Kris 风控否决 (单笔金额超过上限 100,000)

LangGraph 自动回到 Zoe 重生成

但条件没变 -> Zoe 又出同样的单 -> Kris 又否

达到 max_retry=2 后, 流程被熔断终止

场景：风控拦截与重试熔断

```
python main.py --stock 600519.SH --capital 200000 --auto-approve
```

注意只改了 --capital 一个参数, 其他跟 CASE-A 一样, 但执行路径完全不同。

=> Conditional Edge 的作用: 同一份代码, 不同路径。

原理：Conditional Edge + 循环回路

1) 路由函数: 决定 Kris 之后去哪

Kris 节点跑完后, 不是固定走到 Human, 而是根据 risk_verdict 走三种路径之一:

verdict.decision	retry_count vs max_retry	下一节点
approve 或 warn	无关	human (走 happy path)
reject 或 halt	retry < max_retry	zoe_retry (回到 Zoe 重生成)
reject 或 halt	retry >= max_retry	END (熔断终止)

原理：Conditional Edge + 循环回路

2) 重试链路图

Step1, Zoe BUY 100 股 @ 1407 -> Kris [REJECT] 单笔金额超限 -> retry_count = 1 -> 回到 Zoe

Step2, Zoe BUY 100 股 @ 1407 -> Kris [REJECT] 同上 -> retry_count = 2 -> 回到 Zoe

Step3, Zoe BUY 100 股 @ 1407 -> Kris [REJECT] 同上 -> retry_count = 2 >= max_retry

StepX, 路由函数返回 END -> 整个图终止

原理: Conditional Edge + 循环回路

3) 关键设计: zoe_retry 中转节点

路由函数不能改 state -- 它只读不写。但我们要"每次重试给 retry_count 加 1", 怎么办?

方案: 专门做一个微型节点 zoe_retry, 唯一职责就是 `retry_count += 1`:

```
kris -. reject + retry < max .-> zoe_retry -> zoe -> kris -> ...
```

↑

└ 只改 `retry_count`, 不做别的

常见错误: 把 改 state 和 路由 混在一个函数里。

LangGraph 强制你分开: 路由函数纯函数 (输入 state, 输出 next_node); 改 state 走真正的 Node。

=> 这种 职责分离 是 LangGraph 编排清晰的秘诀。

原理：Conditional Edge + 循环回路

4) 状态在重试中是怎么传递的?

注意一个细节: Kris 否决时会把建议仓位上限塞进 verdict (suggested_max_pct), 下一轮 Zoe 会读取它来缩量重发。这是 State 在循环中传递信息的典型用法:

第 1 轮 Zoe -> trade_signal: 100 股

Kris -> risk_verdict: REJECT, suggested_max_pct=0.7 # Kris 建议降仓到 70%

(回到 Zoe)

第 2 轮 Zoe 读 last_verdict -> 看到 suggested_max_pct=0.7

Zoe -> trade_signal: 70 股 # 缩量重发

Kris -> risk_verdict: 可能这次就 PASS 了

本 CASE 里 Kris 触发的是"单笔金额上限"规则 (REJECT 而非 WARN), 没有给 suggested_max_pct, 所以 Zoe 第二轮还是出同样的 100 股 -> 还是被否 -> 最终熔断。

代码逻辑

1) 路由函数

```
def route_after_kris(state: dict) -> str:
    """
    Kris 否决 -> 重试上限内回到 Zoe, 否则结束
    Kris 通过 -> 走人在回路
    """
    verdict = state.get("risk_verdict", {})
    retry = int(state.get("retry_count", 0))
    max_retry = int(state.get("max_retry", 2))
    decision = verdict.get("decision", "approve")
    if decision in ("halt", "reject"):
        if retry >= max_retry:
            print(f"[Graph] Kris 连续否决达上限 ({retry}/{max_retry}), 终止流程")
            return END
        print(f"[Graph] Kris 否决, 重试 {retry + 1}/{max_retry} -> 回到 Zoe")
        return "zoe_retry"
    return "human"
```

代码逻辑

2) zoe_retry 中转节点

```
# 重试中转节点: 累加 retry_count (不能放在 route 里, 路由函数不能改 state)
def zoe_retry_bump(state: dict) -> dict:
    return {"retry_count": int(state.get("retry_count", 0)) + 1}
```

3) 图编排 (Conditional Edge + 循环)

```
g.add_node("zoe", zoe_node)
g.add_node("zoe_retry", zoe_retry_bump)
g.add_node("kris", kris_node)
g.add_node("human", human_review_node)

g.add_edge("zoe", "kris")
```

```
# Kris 出口 -> 三选一 (条件边)
g.add_conditional_edges(
    "kris",
    route_after_kris,
    {"zoe_retry": "zoe_retry", "human": "human", END: END},
)
g.add_edge("zoe_retry", "zoe") # 形成循环: kris -> zoe_retry -> zoe ->
kris -> ...
```

graph.py

代码逻辑

4) Kris 触发 单笔金额上限 的规则

```
class PreTradeGuard:
    def check_order_amount(self, order: Order) -> RiskDecision:
        """规则1: 单笔金额上限"""
        if order.amount > self.max_order_amount:
            return RiskDecision(
                decision=Decision.REJECT,
                reason=f"单笔金额 {order.amount:,.0f} 超过上限 {self.max_order_amount:,.0f}",
                rule_name="单笔金额上限",
            )
        return RiskDecision(decision=Decision.APPROVE, ...)
```

代码逻辑

Thinking: max_order_amount 怎么算的?

变量	值	说明
capital	200,000	用户传入的资金
max_order_amount	100,000	= max(capital * 0.5, 100_000) = max(100k, 100k)
price	1407.24	茅台最新收盘价
quantity	100	Zoe 最小试探单 1 手
amount	140,724	= 1407.24 × 100, 超过 max_order_amount

Kris 节点的 max_order_amount 配置, 来自 nodes/kris_node.py:

```
kris = RiskManager(config={
    "pre_trade": {
        "max_order_amount": max(capital * 0.5, 100_000), # 关键
        ...
    },
})
```

Summary

运行结果

python main.py --stock 600519.SH --capital 200000 --auto-approve

[Charles] 立场: bullish | 信心: 0.82

[Charles] 核心: 贵州茅台进入市场化改革兑现期, 直销占比跃升与非标酒高溢价驱动盈利质量

提升, 2026年净利润增速有望达 5%+, 估值中枢上移至 33x PE

[Charles] 催化剂 3 条 | 风险 3 条

[Charles] 研报附件: 600519_SH_20260418_134243.html

[Zoe] MACD 回测 2025-04-08 ~ 2026-04-17 | 胜率 38.5% | 最新信号 death_cross @ 1407.24

[Zoe] 算出仓位不足 1 手, 按最小试探单 100 股出单 (交给 Kris 审批)

[Zoe] 决策: BUY 100股 @ 1407.24 | 分歧: Charles 强势 bullish, MACD death_cross -> 小仓试探

[Kris] 风控官开始审批

[Kris] 600519.SH ATR(14) = 27.199

[Kris] [REJECT] 综合审批: [单笔金额上限] 单笔金额 140,724 超过上限 100,000

[Graph] Kris 否决, 重试 1/2 -> 回到 Zoe

[Zoe] 信号官开始工作 -- 标的: 600519.SH # <-- 第二轮

[Zoe] (同样的输入, 同样的输出)

[Zoe] 决策: BUY 100股 @ 1407.24

[Kris] [REJECT] 综合审批: [单笔金额上限] 单笔金额 140,724 超过上限 100,000

[Graph] Kris 否决, 重试 2/2 -> 回到 Zoe

Summary

运行结果

=====

[Zoe] 信号官开始工作 -- 标的: 600519.SH # <-- 第三轮

=====

[Zoe] 决策: BUY 100股 @ 1407.24

[Kris] [REJECT] 综合审批: [单笔金额上限] 单笔金额 140,724 超过上限 100,000

[Graph] Kris 连续否决达上限 (2/2), 终止流程

=====

团队 workflow 执行汇总

=====

标的: 600519.SH

可用资金: 200,000

[Charles] bullish (信心 0.82) -- 贵州茅台进入市场化改革兑现期...

[Zoe] buy 100 股 @ 1407.24 -- 分歧: Charles 强势, MACD death_cross

[Kris] reject -- [单笔金额上限] 单笔金额 140,724 超过上限 100,000

[Human] approved = None # <-- 没走到人审

[Trader] # <-- 没走到下单

--- 节点对话历史 ---

[13:42:43] charles | bullish (0.82) -- 贵州茅台进入市场化改革兑现期...

[13:42:46] zoe | buy 100股 @ 1407.24

[13:42:49] kris | reject -- 单笔金额 140,724 超过上限 100,000

[13:42:51] zoe | buy 100股 @ 1407.24 # 重试 1

[13:42:53] kris | reject -- 单笔金额 140,724 超过上限 100,000

[13:42:55] zoe | buy 100股 @ 1407.24 # 重试 2

[13:42:57] kris | reject -- 单笔金额 140,724 超过上限 100,000

打卡：风控拦截与重试熔断



以贵州茅台 600519.SH、资金只有 20 万、自动放行人审为例 (会触发风控):

观察风控否决之后, 流程如何退回 Zoe 缩量重试, 以及上限到达后如何熔断:

- 同样跑完 Charles → Zoe → Kris; 资金偏小时, Kris 会判 reject (例如单笔金额超限)。
- 风控否决后, 流程不直接结束, 而是回到 Zoe 节点, 让 Zoe 缩小仓位重新出单。
- 每次重试前先在一个微型节点把重试次数 +1, 留下计数痕迹。
- Zoe 调小仓位重发, Kris 再次审批; 这条 Kris → Zoe 的环最多走 2 次 (默认重试上限)。
- 若 Kris 仍连续否决, 触发熔断: 流程直接结束, 不进入人审, 也不下单。

场景：人在回路真人交互

场景：人在回路真人交互

TO DO: 以 600519.SH (贵州茅台)为例进行操作

- 资金: 100 万 (足够下单)
- 差异: 不带 --auto-approve 参数
- 预期行为:

Charles -> Zoe -> Kris 跑完 (跟 CASE-A 一样)

到 human 节点, 流程暂停, 把待审订单展示给用户

用户在终端输入 yes 或 no

程序 resume, 根据用户决定走 trader 或 END

场景：人在回路真人交互

```
python main.py --stock 600519.SH --capital 1000000
```

跑到人审时会卡住, 等你输入:

是否授权下单 (yes/no): yes

=> 输入 yes -> 走 trader 下单

=> 输入 no 或回车 -> 走 END (放弃下单)。

原理：interrupt + Command(resume) 双轴控制

Thinking：为什么这个特性这么重要？

在 Agent 模式 (nanobot/deepagents) 里, 必须人类授权才下单 => 只能靠 prompt 提醒 LLM:

```
## 交易规则 (必须严格遵守)
```

1. AI 提供建议方案, 人类授权执行 -- 用户明确授权后才执行下单
2. ...

问题: LLM 可能忘, 可能误解上下文, 可能被新指令覆盖。

=> 在合规要求下这是不可接受的。

Workflow 模式用 interrupt 把这个约束从软提示 => 变成 硬卡点

- 没有 Command(resume=...) 它就是不会往下走。
- state 还会被 Checkpointer 存好, 崩溃重启都能恢复。

原理：interrupt + Command(resume) 双轴控制

组件	职责
Checkpointter	每次节点跑完, 把 state 存进去 (用 thread_id 索引)
interrupt(payload)	节点内调用, 把 payload 抛回上层调用者, state 保留
Command(resume=value)	上层把人类回复传回去, interrupt 调用返回这个 value, 节点继续执行
thread_id	同一个会话用同一个 thread_id, 让 Checkpointer 找到正确的 state

原理: interrupt + Command(resume) 双轴控制

暂停-恢复的完整流程:

Step1, `graph.invoke(initial_state, config={thread_id: "x"})` -- 启动图

Step2, `charles -> zoe -> kris` 三个节点跑完, state 被 Checkpointer 持久化

Step3, `human_review_node` 内部调 `interrupt(payload)` -- 整个图 " 返回 ", payload 被存到 `state.tasks[0].interrupts`

Step4, 调用方 `graph.get_state(config)` -> 看到 next 包含 "human", 知道流程被卡了

Step5, 调用方从 `snapshot.tasks[0].interrupts[0].value` 取出 payload, 展示给用户

Step6, 用户输入 "yes"

Step7, 调用方 `graph.invoke(Command(resume="yes"), config={thread_id: "x"})` -- 关键: 同一个 thread_id

Step8, `human_review_node` 内 `interrupt()` 调用返回 "yes", 节点继续执行, 写 `state["approved"] = True`

Step9, 路由到 `trader -> 下单 dry-run -> END`

原理：interrupt + Command(resume) 双轴控制

Thinking: 第 6 步的用户输入一定是命令行 input 么？

不一定

- 本 CASE 用 input() 是为了演示方便
- 生产系统可以是 FastAPI 接口, 等前端 POST 一个 yes/no
- 也可以是企业微信机器人, 用户在群里回复: @bot yes
- 甚至可以是邮件审批 (邮件回复 -> webhook -> resume)

LangGraph 本身不关心 用户怎么回复, 它只管: 我被 resume 了, 拿到 value 继续走。

代码逻辑：human 节点

1) human 节点 -- interrupt 的写法

```
from langgraph.types import interrupt

def human_review_node(state: dict) -> dict:
    view = state.get("investment_view", {})
    signal = state.get("trade_signal", {})
    verdict = state.get("risk_verdict", {})

    # 1) 把 Charles+Zoe+Kris 的三方信息汇总成 payload
    payload = {
        "type": "trade_approval_request",
        "stock": signal.get("stock_code"),
        "direction": signal.get("direction"),
        "quantity": signal.get("quantity"),
        "price": signal.get("price"),
```

```
        "amount": signal.get("price", 0) * signal.get("quantity", 0),
        "charles_stance": view.get("stance"),
        "charles_confidence": view.get("confidence"),
        "charles_summary": view.get("summary"),
        "charles_report_html": view.get("report_html_path"), # <--
Artifact 路径
        "zoe_reason": signal.get("reason"),
        "zoe_winrate": signal.get("backtest_winrate"),
        "kris_decision": verdict.get("decision"),
        "kris_reason": verdict.get("reason"),
        "prompt": "是否授权下单? 回复 yes / no",
    }
```

[nodes/human_node.py](#)

代码逻辑：human 节点

2) 关键: interrupt() 抛出 payload, 整个图暂停

```
user_decision = interrupt(payload)
```

3) resume 之后, user_decision 就是 Command(resume=...) 传进来的值

```
if isinstance(user_decision, dict):
```

```
    approved = bool(user_decision.get("approved", False))
```

```
    note = str(user_decision.get("note", ""))
```

```
else:
```

```
    text = str(user_decision).strip().lower()
```

```
    approved = text in ("y", "yes", "ok", "approve", "1", "true")
```

```
    note = "" if approved else "用户拒绝"
```

```
return {"approved": approved, ...}
```

[nodes/human_node.py](#)

代码逻辑：怎么处理 interrupt 和 resume

2) 调用方 -- 怎么处理 interrupt 和 resume

```
from langgraph.types import Command

def run_workflow(stock, capital, auto_approve, question):
    graph = build_graph(with_checkpointer=True)
    thread_id = f"run-{stock}-{os.getpid()}"
    config = {"configurable": {"thread_id": thread_id},
"recursion_limit": 50}

    # === 第一段: 跑到 interrupt ===
    state = graph.invoke(initial_state, config=config)

    # === 检查是否被 human_review interrupt 卡住 ===
    snapshot = graph.get_state(config)
    next_nodes = snapshot.next
```

```
if next_nodes and "human" in next_nodes:
    # 拿出 interrupt 的 payload 给用户看
    interrupts = snapshot.tasks[0].interrupts if snapshot.tasks else []
    if interrupts:
        payload = interrupts[0].value
        print("[人在回路] 待用户授权下单:")
        for k in ("stock", "direction", "quantity", "price", ...):
            print(f" {k}: {payload.get(k)}")

    if auto_approve:
        user_reply = "yes"
    else:
        user_reply = input("是否授权下单 (yes/no): ").strip() or "no"

    # === 第二段: resume 继续执行直到结束 ===
    state = graph.invoke(Command(resume=user_reply), config=config)
    return state
```

main.py

代码逻辑

3) 路由函数 -- approved 决定下一步

```
def route_after_human(state: dict) -> str:
    """人类授权 -> 下单; 否则结束"""
    return "trader" if state.get("approved") else END

g.add_conditional_edges("human", route_after_human,
    {"trader": "trader", END: END})
```

4) Artifact 路径 -- 让人审能直接打开 HTML 研报

注意 payload 里有一个 charles_report_html 字段, 这是 Charles 研报附件的磁盘路径。

=> 人审看到后可以直接在浏览器打开评阅, 不必把整个研报塞进 state:

```
charles_report_html : outputs/reports/600519_SH_20260418_133935.html
```

Thinking: Artifact vs State 怎么选?

- State: 短小、结构化、要参与路由判断的 -- stance / confidence / 路径
- Artifact: 大块、原始格式、给人评阅或归档的 -- 几千字研报 / PDF / 图表

把研报塞进 state 会让 Checkpointer 存储膨胀, 还会污染日志输出。

运行结果

运行结果：用户输入 yes (放行)

python main.py --stock 600519.SH --capital 1000000

[Charles] 立场: bullish | 信心: 0.85

[Charles] 研报附件: 600519_SH_20260418_134721.html

[Zoe] 决策: BUY 100股 @ 1407.24

[Kris] [PASS] 综合审批: 所有检查通过

=====

[人在回路] 待用户授权下单:

=====

stock : 600519.SH

direction : buy

quantity : 100

price : 1407.24

amount : 140724.0

charles_stance : bullish

charles_confidence : 0.85

charles_summary : 茅台 2026年Q1 业绩确定性高增长...

charles_report_html :

outputs/reports/600519_SH_20260418_134721.html

zoe_reason : 分歧: Charles 强势 bullish, MACD death_cross

-> 小仓试探

zoe_winrate : 38.5

kris_decision : approve

kris_reason : 所有检查通过

是否授权下单 (yes/no): yes # <-- 用户在这里输入

运行结果

[Human] 用户决定: APPROVED
[Trader] [DRY-RUN] BUY 600519.SH 100股 @ 1407.24

=====

团队 workflow 执行汇总

=====

[Charles] bullish (信心 0.85) -- 茅台 2026年Q1 业绩确定性高增长...
[Zoe] buy 100 股 @ 1407.24 -- 分歧: ...
[Kris] approve -- 所有检查通过
[Human] approved = True
[Trader] dry-run: buy 100 @ 1407.24

运行结果

运行结果：用户输入 no (拒绝)

是否授权下单 (yes/no): no # <-- 用户拒绝

[Human] 用户决定: REJECTED 用户拒绝

=====

团队 workflow 执行汇总

=====

[Charles] bullish (信心 0.85) -- 茅台 2026年Q1 业绩确定性高增长...

[Zoe] buy 100 股 @ 1407.24 -- 分歧: ...

[Kris] approve -- 所有检查通过

[Human] approved = False

[Trader] # <-- trader 没被调用

Summary

观察项	值
graph.invoke 被调用了几次	2 次 (启动一次 + Command(resume=...) 一次)
第一次 invoke 返回时 state.next	("human",) -- 表示卡在 human 节点
interrupt 的 payload 在哪	snapshot.tasks[0].interrupts[0].value
resume 用同一个 thread_id	是 (Checkpoint 靠它找到正确的 state)
recursion_limit	50 (足够 5 节点 + 重试 + resume)

Summary

- interrupt 是合规系统的硬需求

所有金融 / 医疗 / 法律 / 安全场景都有：决策前必须人工审核 的硬要求。

- Agent 模式: 靠 prompt 写必须人审 -- LLM 可能忘, 不可审计
- Workflow 模式: interrupt 是代码强制, 没人 resume 它就是不会往下走

=> 软约束 vs 硬约束 的差异。

Summary

- Checkpointer 让中断不是结束

关键点: interrupt 之后整个 Python 进程可以退出, state 还安全地存在 Checkpointer 里。

下次想恢复, 只要拿同一个 thread_id 调 resume 就行。

这里用的是 MemorySaver (内存), 进程退出 state 就丢了, 方便演示。

生产可以换:

Checkpointer	持久化	适用场景
MemorySaver	内存 (进程级)	一次性脚本
SqliteSaver	本地 .db 文件	单机生产 / 桌面应用
PostgresSaver	Postgres	分布式生产 / 多实例

Summary

- human 节点的 payload 是一个决策仪表盘

注意我们的 payload 不只是"yes/no?", 而是把 Charles+Zoe+Kris 三方观点全部展示:

- Charles 立场 + 信心 + 摘要 + 完整研报路径
- Zoe 信号 + 历史胜率 + 决策原因
- Kris 风控决议 + 拒绝原因

⇒ 这是人在回路的价值:

- 把多方信息汇总, 让人做判断。
- Workflow 模式天然支持这种汇总, 因为 state 里啥都有。

Summary

- 把 input() 换掉就是真正的产品

本 CASE 用 input(“是否授权下单: ”) 是为了方便演示。

把这个换成下面任何一个, 你就有了一个生产可用的人审系统:

```
# 方案 A: HTTP API
@app.post("/approve/{thread_id}")
def approve(thread_id, decision: str):
    state = graph.invoke(Command(resume=decision),
                           config={"configurable": {"thread_id": thread_id}})
    return state
```

interrupt + Command(resume) 让 LangGraph 流程具备人随时可介入的能力 => 合规与安全的保障

```
# 方案 B: 微信机器人 (用户在群里发 "/approve thread123 yes")
async def on_message(msg):
    if msg.text.startswith("/approve"):
        _, tid, decision = msg.text.split()
        graph.invoke(Command(resume=decision),
                      config={"configurable": {"thread_id": tid}})
```

方案 C: 邮件审批

"""

用户邮件回复 -> webhook -> 解析回复内容 -> resume

"""

打卡：人在回路



以贵州茅台 600519.SH、资金 100 万、不带自动放行为例：

体验 LangGraph 的硬性人审卡点：没有人类输入，流程绝不会自己往下跑去下单：

- 前段跑完 Charles → Zoe → Kris, 顺利通过风控。
- 进入人审节点时, 终端打印订单摘要 (标的 / 方向 / 数量 / 价格 / 三方理由), 流程在此暂停。
- 命令行提示「是否授权下单 (yes/no)」, 由你真人输入, 此时 State 已被自动存档, 即便重启也能续跑。
- 输入 yes → Trader 执行下单 (默认模拟盘); 输入 no → 流程直接结束, 不下单。



Thank You
Using data to solve problems

